

專案報告：台灣農業氣象預報系統

1. GitHub Repository

專案的原始碼已上傳至 GitHub：<https://github.com/bonerdog0507/HW2-Web-App>

2. Live Demo

網頁應用程式可透過 Streamlit Community Cloud 即時預覽：[Live Demo on Streamlit](#) (註：此為預留的 Streamlit 部署網址)

3. Demo 圖片

以下為應用程式在本地端運行的實際截圖：



4. Development Log (開發日誌)

本專案旨在提供一個即時、互動且高效的台灣農業氣象預報觀測平台。以下為詳細的開發歷程與技術決策記錄：

第一階段：專案初始化與 API 探勘

- 環境建置與相依性管理：**專案初期，我們首先確立了技術堆疊 (Python、Streamlit、Folium、SQLite3)。為了確保環境隔離與可重製性，建立了 Python 虛擬環境 (venv) 並撰寫 requirements.txt 以明確規範 requests, pandas, streamlit, folium 以及 streamlit-folium 的版本相依。
- CWA API 串接與分析：**在對中央氣象署的開放資料平台進行調研後，我們選擇了「農業氣象預報 (F-A0010-001)」端點。開發初期遇到了 SSL 憑證驗證失敗的問題，透過在 requests.get 中設定 verify=False 並結合 urllib3.disable_warnings 成功克服。接著透過撰寫測試腳本，層層解析深層嵌套的 JSON 結構 (cwaopendata -> resources -> resource -> data -> agrWeatherForecasts -> weatherForecasts -> location)，確定了資料萃取的精確路徑。

第二階段：資料擷取與後端持久化儲存 (scraper.py)

- 資料萃取邏輯實作：**在爬蟲模組中，我們設計了過濾機制，精準抓取「北部地區」、「中部地區」、「南部地區」、「東北部地區」、「東部地區」與「東南部地區」這六大關鍵區域。透過迭代解析 weatherElements 中的

MaxT（最高溫）與 MinT（最低溫）陣列，將未來七天的每日溫度預報與對應日期成對提取出來。

- **從 CSV 到 SQLite3 的架構升級：**原先的設計是將爬取結果匯出為 `weather_data.csv`。然而，考量到未來資料擴充性、查詢效能以及結構化關聯的需要，我們在後續將儲存層升級為 SQLite3 關聯式資料庫。
- **資料表結構設計：**使用 `sqlite3` 模組自動建立名為 `data.db` 的資料庫檔案，並規劃了 `TemperatureForecasts` 資料表。其結構包含了：
 - `id` (INTEGER PRIMARY KEY AUTOINCREMENT)：確保每筆資料的唯一性。
 - `regionName` (TEXT)：儲存區域中英文名稱。
 - `dataDate` (TEXT)：記錄預報日期 (YYYY-MM-DD)。
 - `mint` (REAL) 與 `maxt` (REAL)：使用浮點數儲存氣溫，方便後續在前端進行平均溫度的數學運算。在每次爬蟲執行前，實作了 `DELETE FROM TemperatureForecasts` 以確保不會累積重複的過期資料，並透過 `executemany` 進行批次插入，提升寫入效能。

第三階段：互動式網頁介面與資料視覺化 (`app.py`)

- **Streamlit 佈局設計：**使用 `st.set_page_config(layout="wide")` 將版面設定為寬螢幕模式，並將主要內容分為左右雙欄 (`st.columns([1, 1])`)，提供使用者並排對照地理位置與量化數據的絕佳體驗。
- **資料存取層整合：**透過 `@st.cache_data` 裝飾器，實作了資料庫讀取的快取機制，避免在使用者互動時頻繁發送 SQL 查詢，顯著提升了前端的渲染速度。運用 `pandas.read_sql_query` 直接從 `data.db` 撈取資料，並在 `DataFrame` 中動態計算出「平均溫度 (AvgT)」。
- **地理資訊系統 (GIS) 視覺化：**
 - **Folium 地圖初始化：**將地圖中心點設定在台灣中心 (緯度 23.5，經度 121.0)，並選用簡潔的 `CartoDB positron` 底圖以凸顯標記點。
 - **動態標示與顏色編碼：**為六大區域定義了大致的經緯度座標字典。程式會遍歷篩選後的數據，並根據平均氣溫自動分配顏色指標 (`<20°C` 為藍色、`20-25°C` 為綠色、`25-30°C` 為黃色、`>30°C` 為紅色)，將其繪製為 `CircleMarker`。
 - ****互動彈出視窗 (Popup)**：**透過嵌入 HTML 與 CSS 樣式，當使用者點擊地圖標記時，會彈出包含地區名稱、最高/最低與平均溫度的精美資訊卡片。
- ****互動式資料篩選 (Dashboard)**：**在右側版面建置了下拉式選單 (`st.selectbox`)，自動提取資料庫中的所有不重複日期供使用者選擇。一旦日期變更，左側的地圖顏色分佈與右側的 `DataFrame` 資料表 (`st.dataframe`) 皆會即時同步刷新。

第四階段：自動化流程與專案部署

- ****一鍵執行腳本 (`run.sh`)**：**為了降低執行門檻，我們撰寫了 Shell Script。該腳本依序執行虛擬環境建立 (`venv`)、相依套件安裝、執行爬蟲 (`python scraper.py`) 抓取最新資料並建立資料庫，最後啟動網頁伺服器 (`streamlit run app.py`)，實現了完全自動化的啟動流程。
- **除錯與穩定性提升：**在開發 `Popup` 的 HTML 字串時，遭遇了 Python f-string 三引號中夾雜反斜線 (`\`) 所引起的 `SyntaxError`。我們透過調整字串格式化邏輯迅速排除了該語法錯誤，確保程式在各環境下皆能穩定執行。
- **版本控制與報告產出：**嚴格規範 `.gitignore`，過濾掉 `venv/`、`*.db`、`*.log` 等非源碼檔案，保持 Git 儲存庫的整潔。我們利用 SSH 驗證成功將專案推送到遠端 GitHub，並使用 `npx md-to-pdf` 工具將這份 Markdown 開發報告轉換為高品質的 PDF 格式 (`report.pdf`) 上傳，完成了專案的最後一哩路。